Cfg66 Developer Guide 0.3.0

Chris Ahlstrom
(ahlstromcj@gmail.com)

April 14, 2026

Cfg66 Logo

Contents

List of Figures

List of Tables

1 Introduction

The *Cfg66* library reworks some of the fundamental code from the *Seq66* project ([4]). This work is in preparation for the version 2 of that project, but might also be useful in other applications.

Cfg66 contains the following subdirectories of `src` and `include`, each of which holds modules in a namespace of the same name:

- `cfg`. Contains items that can be used to manage a generic configuration, including application names, settings basics, and an INI-style configuration-file system. Added are data-type indicators and help text.
- `cli`. Provides C/C++ code to handle command-line parsing without needing to use, for example, getopt. While it somewhat matches how getopt works, it also allows combining option sets and provides a parser object that contains the current status of all available options, as well as help text.
- `session`. Contains classes for managing a basic "session". Here, a session is simply a location to put configuration files; multiple locations can be supported. Session filenames are based on a "home" configuration directory, optional subdirectories, and application-specific item names.
- `util`. Contains file functions, message functions, string functions, and other functionality common to all our "66" application and libraries.

In the sections that follow, the basic are described. At some point we will make the effort to add some *Dia* diagrams to make the relationships more clear.

1.1 Naming Conventions

Cfg66 uses some conventions for naming things in this document.

- `$prefix`. The base location for installation of the application and its ancillary data files on *UNIX/Linux/BSD*:
  - `/usr/`
  - `/usr/local/`
- `$winprefix`. The base location for installation of the application and its ancillary data files on *Windows*.
  - `C:/Program Files/`
  - `C:/Program Files (x86)/`
- `$home`. The location of the user's configuration files. Not to be confused with `$HOME`, this is the standard location for configuration files. On a UNIX-style system, it would be `$HOME/.config/appname`. The files would be put into a `po` subdirectory here.
- `$winhome`. This location is different for *Windows*: `C:/Users/user/AppData/Local/PACKAGE`.

1.2 Future Work

- Hammer on this code in *Windows*.

2 Cfg Namespace

This section provides a useful walkthrough of the `cfg` namespace of the *cfg66* library. Here are the classes (or modules) in this namespace:

- `cfg::appinfo`
- `cfg::basesettings`
- `cfg::comments`
- `cfg::configfile`
- `cfg::history`
- `cfg::inifile`
- `cfg::inimanager`
- `cfg::inisection`
- `cfg::inisections`
- `cfg::memento`
- `cfg::options`
- `cfg::palette`
- `cfg::recent`

2.1 cfg::appinfo

The `cfg::appinfo` structure encapsulates basic information about an application:

- *cfg::appkind*. Indicates if the application is a headless application (such as a daemon), a command-line, *ncurses*, GUI, or test application. In some cases it can be useful to know how the application is running.
- *App name*. Provides the short name of the application, which can be shown in warning messages or be used to name the application as a node, for example, in *JACK*.
- *App version*. Provides a version number, or optionally, a variant on the *Name* plus the version number.
- *Main config section name*. Holds the name of main configuration section (usually present only in the "session" or "rc" file). The default value is "[Cfg66]".
- *App home configuration directory*. Provides the name of the configuration directory for the application, such as `/home/user/.config/appname`. Modified by the `-home` option.
- *Home configuration file*. Provides the name of the main configuration file, such as `appname.rc`. Modified by the `-config` option.
- *Client name*. Either the short application name or a variant of it, useful in session managers, for example. An example under *NSM* would be `seq66v2.nUKIE`.
- *App tag*. The short application name with the version number tacked on.
- *Arg0*. Holds the complete path to the executable as determined at run-time.
- *Package name*. Provides the name of the package, which could be the same as the application or library name.
- *Session Tag*. Provides the session name for the application; it can be the application name, a modification of the application name, or something completely different, such as a session name given by a user. Useful in long error/warning/info messages.
- *App icon*. Provides the base name of the application icon, if not empty.
- *Version text*. Reconstructed version information for the application.
- *API engine*. Some applications might use various libraries. For example, for a MIDI application it might be one of these MIDI libraries: `rtmidi`, `rtl66`, or `portmidi`.
- *API version*. Indicates which version of an API is in force. In some cases this can be detected at run-time.
- *GUI version*. Indicates the GUI or "curses" version, such as *Qt 6.1* or *Gtkmm 3.0*.
- *Short client name*. Similar to *client name*, but never has anything appended to it.
- *Client name tag*. Provides the name to show on the console in error/warning/info messages, the short client name surrounded by brackets, such as `[seq66v2]`.

All of these items can be set at once using the `appinfo` constructor that has a large number of parameters. In addition, there are a many "free" functions in the `cfg` namespace for setting and getting these values. See `appinfo.hpp` for a summary.

2.2 cfg::basesettings

THe `cfg::basesettings` class is a base class. It provides common settings useful in any application:

- *File name*. Holds the (optional) name of the file that holds the settings data.
- *Ordinal version*. Provides a simple integer indicating the version of the file, useful in adapting to changes in settings.

- *Modify/unmodify function and "modified" flag.* Indicates if the settings have been modified.
- *Config format.* A free-form string indicating the format of the data, such as `INI`, `XML`, or `JSON`. For "66" libraries and applications, the `INI` format is sufficient.
- *Config type.* A short string that indicates something about the format or content of the file. For example, in Seq66, common values were 'rc' and 'usr', with these values also representing the file extension.
- *Ordinal version.* A simple integer that is incremented each time a change is made in the configuration format or values-present that isn't detectable during parsing.
- *Comments.* An object holding the main comments that describe something about the settings file.
- *Is error.* Indicated if there was a error during parsing.
- *Error message.* If an error occurred, the error message is stored for display.

2.3 cfg::comments

`cfg::comments` holds a string describing something general about the configuration, meant to be included as the text of a `[comment]` section. Provides some setter and getter functions. It is a simple class.

2.4 cfg::configfile

The `cfg::configfile` class is an `abstract base class`. It provides some items common to configuration files, including an extensive set of functions to parse sections and configuration variables in an INI format in a line-by-line fashion. The member functions `parse()` and `write()` are *pure virtual*, and must be overridden in a derived class. A good example is `cfg::inifile`.

These are the main externally-accessible values:

- *File extension.* Common values are 'rc' and 'session'.
- *File name.* Provides the file name, normally as a full-path file-specification.
- *File version.* Provides the current version of the derived configuration file format. Set in the constructor of the configfile-derived object, and incremented in that object whenever a new way of reading, writing, or formatting the configuration file is created.

Also too many to list, but it includes functions such as `get_integer()` and `write_integer()`.

These work by having the whole file read into an `std::ifstream`, and then searching the string over and over to read all the variables. Sounds inefficient, but in practise it is very fast.

Finally, there are free functions to delete a configuration file and to make a copy of a configuration file.

2.5 cfg::history

One of the things not handled so well in *Seq66* is the undo/redo functionality. The `history` template class implements undo/redo using the `memento` class described below. It follows the *Design Patterns* book ([2]). Also informative is [1]. Also see the `cfg::memento` class below.

The heart of the `history` template is the history list:

```
std::deque<memento<TYPE>> m_history_list;
```

Member functions are provided to see if history entries are undoable/redoable, undo and redo them, check the maximum size of the list, etc. The `history.cpp` module provide a small test of history for the `cfg::options` class.

The `history_test` application provides a more substantive test of history. It provides an `cfg::options::container` with a few options, and then applies `undo` and `redo` operations on them, checking the results.

To do: test range validation.

2.6 cfg::inifile

`cfg::inifile` is derived from the `cfg::configfile` class. It contains a reference to a class that manages multiple INI sections: `cfg::inisections`. It overrides the `cfg::configfile::parse()` and `cfg::configfile::write()` functions. It provides two protected functions, `parse_section()` and `write_section()`.

The `ini_test` application sets up a couple of `cfg::inisection::specification` named structures with a long description and a list of values to be stored. The `cfg::stock_cfg66_data()` and `cfg::stock_comment_data()` specification are defined as well. All section specifications are wrapped in a `cfg::inisections::specification` structure that is used to create a `cfg::inisections` object and then use it to create an INI file for output, exercising `inifile::write()`. The file generated is `tests/data/fooout.rc`.

The same setup is used to exercise `inifile::parse()`. The file read is `tests/data/fooin.rc`. The file generated from `fooin.rc` is `tests/data/fooinout.rc`. The file generated from the test's internal data is `tests/data/fooout.rc`. The results can be found in the `tests/data` directory.

How does it work?

```
cfg::inisections sections(exp_file_data, "fooout");
```

The `exp_file_data` specification provides the configuration type (file extension without the period) `rc`, the putative file directory `./tests/data`, the default base name of the configuration file `ini_test_session`, followed by the description of the INI sections object and a list of the sections in the INI sections object.

Once the `exp_file_data` structure is loaded into the `inisection`, we can hook it to an `inifile` and write the file:

```
cfg::inifile f_out(sections);
success = f_out.write();
```

The section data items are written to the assembled full file-specification, `./tests/data/fooout.rc`.

The next test creates an `inisections` with the base name "fooin". An `inifile` is created from this section and is used to parse `./tests/data/fooin.rc`.

```
cfg::inisections sections(exp_file_data, "fooin");
cfg::inifile f_in(sections);
```

```
success = f_in.parse();
```

Once written, we create another `inifile`, changing it's base-name to "fooinout". Then we write it:

```
cfg::inifile f_inout(sections, "fooinout");  // changes the name
success = f_inout.write();
```

The files `./tests/data/fooin.rc` and `./tests/data/fooinout.rc` should be almost identical.

2.7 cfg::inimanager

`cfg::inimanager` is meant to manage multiple `cfg::inisections` objects. Each `cfg::inisections` holds multiple `cfg::inisection` objects, and represents the settings in an `cfg::inifile`. Each INI file is associated with a unique configuration file type, such as "rc" or "session", and a section name,

`cfg::inimanager` defines a `section` types, which is a a map with the key being a configuration type, such as "rc", and the value being an `cfg::inisections` object.

`cfg::inimanager` also contains a `cli::multiparser` which can map unique option names to the INI file and INI sections to which they belong. Not all options from an INI file will be useful from the command line, though.

The `ini_set_test` application first defines some *global* options peculiar to it: "–list", "–read", "–test", and "–write". These are *global* options because they are not associated with an INI file or an INI section. The `cfg::inimanager` is created with this list, and populates the options. Then two `cfg::inisection` objects are added, one for `cfg::small_data` and one for `cfg::rc_data`.

The caller can then exercise the global options, including the internal, stock options. The `-help` option will show all of the global options, plus all of the sections and options defined for `cfg::small_data` and `cfg::rc_data`. The `-read` and `write` options accept a file-name and process it.

Note: if something like the following message appears, fix the issue. Otherwise, the option cannot be parsed, even if it appears in the help text.

```
[iniset] Couldn't insert <sets-mode,(rc,[misc])>: Change option to a
unique name
```

Let us walk through some runs of `ini_set_test`.

```
$ ./build/test/ini_set_test --help
```

This command emits color-coded help text showing the options, a brief description of each, and the default value of each option. The text is extracted from the setup structures, as provided in the file `./tests/small_spec.hpp`:

```
inisection::specification small_misc_data
inisection::specification small_interaction_data
inisection::specification small_comments (stock comments)
inisection::specification small_cfg66_data (stock [Cfg66] section)
```

```
inisections::specification small_data
```

The last specification collects all of the others and adds directory and descriptions. Also included are all the specifications in the file `tests/rc_spec.hpp`, which we will not show here.

The first part of the help text shows the "global" options, which consist of internal stock options plus some additional test options defined by `cfg::options::container s_test_options` in `tests/ini_set_test.cpp`: "–list", "–read", "–write", and "–tests". The global options are not associated with either an INI file or an INI section.

Following the global options are specific options associated with an INI file and and INI section. Here's a partial example:

```
rc:[misc] A number of miscellaneous options
 --manual-ports           Show real system ports, not 'usr' port names.
                          [false]
 --port-naming=v          Port amount-of-label showing. [short]
```

The "rc" represents the INI file, which can have any name, but with an extension of `.rc`. For a given program, there can be only one INI file with that extension. The "[misc]" represents an INI section in that INI file.

The setup process is fairly simple once all of the INI specification structures have been coded. First, we add the global test options to the stock options. Then we add the `inisections` defined in the test "hpp" files.

```
cfg::inimanager cfg_set(s_test_options);
bool success = cfg_set.add_inisections(cfg::small_data);
if (success)
    success = cfg_set.add_inisections(cfg::rc_data);
```

Next, we need to create a `cli::multiparser` and parse the command-line:

```
cli::multiparser & clip = cfg_set.multi_parser();
success = clip.parse(argc, argv);
```

Now, not all of the global options have been implemented, so we describe only the ones that work.

```
$ ./build/test/ini_set_test --list --verbose
```

The "verbose" option does nothing, but the "list" option shows the current values of all options, and it stars the ones that have been modified from the command line: "verbose" and "list". We can add a "log" option to dump the list to a file:

```
$ ./build/test/ini_set_test --log=t.log --list --verbose
```

The next test option:

```
$ ./build/test/ini_set_test --test
```

That option is currently a minimal implementation. More to come.

```
$ ./build/test/ini_set_test --read=tests/data/fooin.rc
```

Note that the "=" can be replaced by a space.

This option depends on being able to find the the desired INI sections object in the INI manager and finding that it is active (it has entries). This INI sections object can then be used to determine what needs to be found in the INI file.

```
$ ./build/test/ini_set_test --write=t.rc
```

This option writes the current settings to `t.rcv` and `t.small` in the `tests/data` directory.

We can combine a number of options to read an INI file, list the data read to a log file, and write it to another file:

```
$ ./build/test/ini_set_test --list --log test.log
        --read tests/data/ini_set_test.rc --write ini-out.rc
```

The output in `test.log` and `ini-out.rc` should match up well with `tests/data/ini_set_test.rc`.

One more use case. Copy `tests/data/ini_set_test.rc` to a temporary file, `i.rc`. Then edit all of the comments (indicated by the hash mark, '#') so that they are obviously different. Then read that file and write it to another temporary file.

```
$ ./build/test/ini_set_test --read=i.rc --write=j.rc
```

Comparing `i.rc` and `j.rc`, one sees that `j.rc` retains the original comments, which ultimately came from the specification defined in the application. That is, the comments are not modifiable in the 'rc' file. (They aren't modifiable in *Seq66* either.

In the future it would be good to read in the comments as well and store them in the descriptions.

We can combine a number of options to read an INI file, list the data read to a log file, and write it to another file:

```
$ ./build/test/ini_set_test --list --log test.log
   --read tests/data/ini_set_test.rc --write ini-out.rc
```

The output in `test.log` and `ini-out.rc` should match up well with `tests/data/ini_set_test.rc`.

Note: The `cfg::inifile` objects are not dealt with directory by `cfg::inimanager`. They are instantiated by the test application. The `session::manager` class will provide access to the functions that read/write INI files.

2.8 cfg::inisection

`cfg::inisection` contains a `specification` structure that provides the name of a section, it's description, and a container of `cfg::options`. It is represented in a configuration file by a section

or tag name enclosed in brackets: `[output-ports]` as an example. Each section can also define a file extension (e.g. `.rc`) to indicate its locus.

It provides a number of functions to return option setting, help text, and more. Also provided are free functions to return a stock main configuration and a stock comments section.

2.9 cfg::inisections

The `inisections.hpp` file has been split, and no long declares four classes.

`cfg::inisections` defines four types:

- `specref`. `std::reference_wrapper<inisection::specification>`.
- `specrefs`. `std::vector<specref>`.
- `sectionlist`. `std::vector<inisection>`.
- `specification`. This structure holds the configuration file's extension, directory, base name, description, and a vector of `specrefs`.

`cfg::inisections` holds a list of `inisection` objects. It represents all of the files, and all of the options that are held by the files. The options are in a single list, and the INI items look them up by name.

Functions are provided to pass along to the `cfg::inisection` objects and to look up INI sections and options. The various parts of a file-name (path, base-name, and file extension) are held, and can be used to get the full file-specification for reading and writing an INI file.

There is a lot to this module. For now, see the comment in the `.hpp` and `.cpp` files.

2.10 cfg::memento

The `cfg::memento` template class is a small object that holds one copy of a state object. It provides these functions:

- `memento (const TYPE & s)`.
- `bool set_state (const TYPE & s)`.
- `const TYPE & get_state () const`.

The `cfg::history` class stores a "history list": `std::deque<memento<TYPE»`.

2.11 cfg::options

The `cfg::options` holds a number of items needed for the specification, reading, and writing of options. The options can be read from configuration file or from the command-line. These items are nested in the class:

- *Static data*. Flags and numbers are provided to indicate if the option is enabled and how it is to be output in a nice format into an INI file.
- *kind*. Indicates if the option is boolean, numerical, a filename, list, string and some others. This enumeration makes it easier to process the option.

- *spec*. This nested structure contains these values. For brevity, the `option_` portion of the name and the type are not shown:
  - `code`. Optional single-character name.
  - `kind`. Is it boolean, integer, string...?
  - `cli_enabled`. Normally true; false disables.
  - `default`. Either a value or "true"/"false".
  - `value`. The actual value as parsed.
  - `read_from_cli`. Option already set from CLI.
  - `modified`. Option changed since read/save.
  - `desc`. A one-line description of option.
  - `built_in`. This option is present in all apps.
- *option*. The `cfg::options::option` is a simple pair of the name of the option and the `spec` that describes it.
- *container*. The `cfg::options::container` is a map (dictionary) of option `specs` keyed bythe name of the option.

Also specified are the name of the source file and the name of the source section in that file.

Included are quite a number of functions for looking up option values and option characteristics. Also include are free functions to make options.

The `options.cpp` module not only contains many comments explaining the module, but also a statically-initialized list of default options that any application can use. It is also a good example of how to create a list of options.

2.11.1 cfg::options::kind

The `cfg::options::kind` enumeration specifies the format of an option variable. Here is summary:

- **boolean**. Each value is either true or false.
- **floating**. The value is either a float or a double.
- **floatpair**. Two floating values in a format like "3.0x4.0".
- **integer**. The value is an integer, whether signed or unsigned.
- **intpair**. Two integer values in a format like "3x4".
- **list**. The value is a multi-line list of items, such as a list of busses or a list of recent files.
- **overflow**. The option is supported by the "–option" option.
- **section**. The INI-section has no variables, but contains some section-specific values. Example: the "[comments]" section, containing paragraphs.
- **string**. The value is a human readable string. It can also be used to hold string versions of enumeration values.
- **filename**. A file-name component, such as a path, a base file-name, or a full file-specification.
- **dummy**. This value is used only when an option cannot be found.

Another type, related to the `section`, but not called out in the list above, is the `comment`.

Option values are retrieved from the command-line via the following functions.

1. `cli::parser::parse()`. Tokens beginning with a `-` or `-` mark an option. The specific option named `-option` is handle as discussed below in section 2.11.1.8 "cfg::options::kind::overflow (–option)" on page 15. Otherwise, the next function is called.

2. `cli::parser::parse_value()`. This function handles single-letter codes and full option names. Next...
3. `cli::parser::extract_value()`. A token such as `-a`, `-a:value`, or `-analyze:value` is tokenized; the first token is returned as the full option name, and the second is the value, if provided after the `:` or `=`. Next...
4. `cli::parser::change_value()`. This function calls the `cfg::options` function of the same name.

Option values are retrieved from a configuration file via the following functions.

1. `cfg::configfile::get_xxx()`. This set of functions calls the next function to get the value string of an option, and then passes it, if needed, to a conversion-from-string function. See the next sections for kind-specific handling.
2. `cfg::configfile::get_variable()`. This function searches for a given option name after first searching for a section name (e.g. `[input-ports]`). Next...
3. `cfg::configfile::extract_variable()`. This function searches the given line for an option name, and returns the value that was found.

The next sections show examples in an INI-style configuration file. In most cases, an equals sign is required. Space before and after is optional, but improves readability.

2.11.1.1 cfg::options::kind::boolean

Boolean options flag either a true value, or, if the option starts with `-no-` (on the command line), a false value. The option format in the configuration file is one of the following:

```
port-naming = true
client-naming = false
```

If the value is not equal to "true", then "false" is assumed. See the `cfg::configfile::get_boolean()`, `cfg::configfile::write_boolean()`, and the `util::string_to_bool()` functions.

2.11.1.2 cfg::options::kind::floating

Here, the value is intended to be a floating-point variable, i.e. of C++ type `float`. The caller might convert it to a longer floating-point type.

```
precision = 0.001
```

See the `cfg::configfile::get_float()` `cfg::configfile::write_float()`, and the `util::string_to_doub` functions.

2.11.1.3 cfg::options::kind::floatpair

This option provides a way to represent 2-dimensional floating-point quantities. They can be separated by a space, a hyphen, a comma, or an x. Here are three examples, which should be followed strictly for readability.

```
flux-parameters = 0.50 1.25
energy-range = 10.0-15.0
```

```
window-scale = 0.5 x 0.5
float-items = 1.0, 8.0
```

See the `cfg::configfile::get_float_pair()` `cfg::configfile::write_float_pair()`, and the `util::string_to_float()` functions.

At some point we should support more than two values on a line.

2.11.1.4 cfg::options::kind::integer

Here, the value is intended to be an integer variable, i.e. of C++ type `int`. The caller might convert it to a longer integer type.

```
string-limit = 32
```

See the `cfg::configfile::get_integer()` `cfg::configfile::write_integer()`, and the `util::string_to_` functions.

2.11.1.5 cfg::options::kind::intpair

This option provides a way to represent 2-dimensional integer quantities. They can be separated by a space, a hyphen, a comma, or an x. Here are three examples, which should be followed strictly for readability.

```
port-pair = 9 10
channel-range = 0-15
window-size = 640 x 400
items = 1, 8
```

Note that other bases are supported via a leading "0" for octal and "0x" for hexadecimal.

See the `cfg::configfile::get_int_pair()` `cfg::configfile::write_int_pair()`, and the `util::string_to_int()` functions.

At some point we should support more than two values on a line.

2.11.1.6 cfg::options::kind::list

A list here is simply a list of non-empty lines, preceded by a count of item. The whole section is just this list; only one list can reside in a section. However, additional options can be held in a list section, as long as they are uniquely named over all of the configuration files in an application.

```
[midi-input]
midi-input-active = true
count = 3
0  1   "[0] 0:1 system:ALSA Announce"
1  1   "[1] 14:0 Midi Through Port-0"
2  0   "[2] 28:0 nanoKEY2 _ CTRL"
```

The whole line is obtained, and the application is responsible for processing it.

Note that the "count" value is not the name of an option, though it can be searched. The list must immediately follow the count. (Blank lines are ignored.)

Extra options (e.g. `midi-input-active`) can be added and handled in the normal way by the color.

See the `cfg::configfile::parse_list()` `cfg::configfile::write_list()`, and the `util::string_to_int(` functions.

See the *recents* section below for a another kind of list.

2.11.1.7 cfg::options::kind::recents

This class holds a list of the most-recent items in a list of items, always strings.

It can be filled from a list, but also includes a couple of true options.

```
[recent-files]
full-paths = false
load-most-recent = true
count = 2
"/home/me/seq66/midi/Shock the Monkey.midi"
"/home/me/seq66/midi/Sitarbeat.midi"
```

2.11.1.8 cfg::options::kind::overflow (–option)

The main purpose of an overflow option is to support additional options on the command line when there are too many of them to be supported by codes in the sets A-Z, a-z, 0-9, etc. Therefore, these options do not appear in a configuration file as overflow options, but as code-less long-name options.

More to come.

2.11.1.9 cfg::options::kind::section

A section is one, such as `[comments]`, that merely contains lines of text, with a line having only a space being considered a line of text. Here's in an example:

```
[description]

This is the first line of the description. The next line
is not blank, but has a space to represent a blank line.

This is the last line of the description.
```

See the `cfg::configfile::parse_section_option()`.

Another type, related, but not called out, is the `comment`, represented by a section name `[comment]`. See the `cfg::configfile::parse_comments()`.

2.11.1.10 cfg::options::kind::filename

2.12 cfg::palette

The `cfg::palette` template class can be used to define a palette of color code pair with a platform-specific color class such as `QColor` from *Qt*. This is a feature copped from *Seq66*.

2.13 cfg::recent

`recent` provides for the management of a list of recently-used items. It is implemented as a deque of strings.

Included are functions to get the most-recent file-name, a file-name by index (with optional removal of the path, for use in menu displays), and functions to manage the list. This is a feature adapted and extended from *Seq66*.

3 Cli Namespace

This section provides a useful walkthrough of the `cli` namespace of the *cfg66* library. In addition, a `C`-only module is provided.

Here are the classes (or modules) in this namespace:

- `cliparser_c`
- `cli::parser`
- `cli::multiparser`

3.1 cliparser_c Module

This module is actually a `C++` module that implements a number of `extern "C"` functions. The functions themselves access an internal and hidden `cli::parser` object and call its member functions to perform the functions.

This module provides a `C` structure that mirrors `options::spec`, plus some free functions to access this structure.

3.2 cli::parser

The `cli::parser` class contains a number of options in a `cfg::options` object. Many of the member functions are pass-alongs to this object. This parser is meant for simple usage; it has no support for accessing configuration files, and supports just one set of options.

3.3 cli::parser / Global Options

The `cli::parser` also provides support for options that can be command to all applications. These stock options are called "global" options, since they are not associated with any configuration file.

- `-version`. Provides the version of the application.

- `-help`. Emits help text. This help text is assembled from the specification structures set up according to the `cfg::options::container`.
- `-description`. This option is meant to show the description fields of the options.
- `-verbose`. Indicates to emit additional information about the run of the application.
- `-inspect`. This option is meant to activate debugging code. The applications determines what this means.
- `-investigate`. This option is meant to activate temporary debugging code for the current purpose desired by the programmer. The applications determines what this means.
- `-log`. This option enables a log file for recording error and information output.

Additional global options can be added, as illustrated in the `ini_set_test` program.

The `parse()` function looks for stock option such as `-help` and `-option log filename`. The `-` sequence terminates a list of options.

It is meant to be similar to `getopt(3)`, but much more flexible and perhaps easier to set up.

The caller can call the following member functions:

- `show_information_only()`. This function returns a boolean value that is true if help, description, or version were requested. Generally, if this information is shown on a command-line, the application does nothing but show the desired information, and exits.
- `version_request()`. Indicates if version information was requested.
- `help_request()`. Indicates if application help was requested. The command-line help text is shown. The caller can also provide further help if desired.
- `description_request()`. Indicates if description information was requested.
- `verbose_request()` and `verbose()`. Indicates if extra output (either command line or via dialog boxes) was requested.
- `inspect_request()`. Indicates if inspection was specified was requested. Generally can be used to output data values as appropriate.
- `investigate_request()`. Indicates if investigation was specified was requested. Defined by the application.
- `use_log_file()` and `log_file()`. Indicates that standard I/O should be redirected to a log file.

There are functions to add, verify, and set option values:

- `add()`.
- `verify()`.
- `set_value()`.
- `change_value()`.
- `clear()`.
- `unmodify()`.
- `unmodify_all()`.

There are also functions to test and retrieve option values:

- *string*. All options are encoded as strings. Strings can be retrieved using `cli::parser::value()`. The default string can be retrieved by `default_value()`.

- *boolean.* `is_boolean()` tests for a value being boolean.
- *other, to do.* There are many more value-related functions in the `cfg::options` class, but they use a reference to a `cfg::options::spec` structure, and can be accessed only by getting the option and spec reference. See section 2.11 "cfg::options" on page 11.

The format of the default value is either a simple string showing the default value, or a *range* string such as "0<=10<=99", where the middle value is the default value.

3.4 cli::parser / Usage

This section provides a walk through the test application `cliparser_test`. It starts with an `appinfo` function to see the name that will appear in console messages:

```
cfg::set_client_name("cli");
cfg::set_app_version("0.2.0");
```

Next, a parser is created via:

```
cli::parser clip(s_test_options);
```

The `s_test_options` list is defined in the `test_spec` header file. Here is an excerpt. Note that we could use an anonymous namespace instead of the keyword `static`.

```
static cfg::options::container s_test_options
{
    {
        {
            "alertable",
            {
                'a', cfg::options::kind::boolean, cfg::options::enabled,
                "false", "", false, false,
                "If specified, the application is alertable.", false
            }
        },
        {
            "canned-code",
            {
                'c', cfg::options::kind::boolean, cfg::options::enabled,
                "true", "", false, false,
                "If specified, the application employs canned code.", false
            }
        },
        . . .
    }
};
```

As an example, this list provides the `-a` and `-alertable` option, which is a boolean option and is enabled. It has a default value of "false", which is set once the list is initialized. The values for "set-from-cli" and "dirty" are false by default at first. After the one-line description, the boolean indicates if the options is a built-in (global, stock) option. That value is set only in the

`global_options()` function in the `options` module. The options in this list are added to the global options.

After construction, the parser can be applied to the command-line arguments:

```
bool success = clip.parse(argc, argv);
```

This can modify (set the value, raise the "set-from-cli" and "dirty" flags) options based on the command-line.

Not all options will have a code, in general, especially if there are a large number of options. Duplicates are checked for. The existing codes can be listed:

```
std::string msg = "Option codes: " + clip.code_list();
```

The command line can also be scanned to see if an option is present. The last parameter indicates if the option is required to exist:

```
bool findme_active = clip.check_option(argc, argv, "find-me", false);
```

Built-in (global) options that make the application show something and then quit can be tested:

```
if (clip.show_information_only())
{
    if (clip.help_request()) . . .
    if (clip.description_request()) . . .
    if (clip.version_request()) . . .
}
```

Some other built-in options:

```
if (clip.verbose_request()) . . .
if (clip.inspect_request()) . . .
if (clip.investigate_request()) . . .
if (clip.use_log_file()) . . .
```

Debug text can be shown; it lists all the options, their values, their default values, and if the value has been modified:

```
std::string dbgtxt = clip.debug_text(cfg::options::stock);
```

Values can also be set or changed by the application itself, rather than from the command-line. It can be retrieved by the long option name or the option code.

```
success = clip.change_value("username", "C. Ahlstrom");
std::string name = clip.value("u");    // or "username"
success = name == "C. Ahlstrom";
```

If an error occurs, which can be checked by a function's return code or by a call to `cli::parser::error_msg()`, then the message can be retrieved for display:

```
std::string errmsg = clip.error_msg();
util::error_message(errmsg);
```

3.5 cli::multiparser

The `cli::multiparser` class extends `cli::parser` in a number of ways, in order to support a suite of command-line options covering mutltiple configuration files.

Support is provided to look up the long option name from an option code character. (The `cfg::options` class also provides this, but it is not directly exposed to `cli::parser`.

```
using codes = std::map<char, std::string>;
codes & code_mappings();
```

This function is used in the `multiparser` override of the the virtual `cli::parser::parse()` function.

In order to support multiple configuration files and multiple configuration sections, we need to way to find out in which file and section an option resides.

```
using duo = struct
{
   std::string config_type;
   std::string config_section;
};
using names = std::map<std::string, duo>;
names & name_mappings();
```

This function is used to find the desired option set, a pointer to the `cfg::options` for a particular file and section.

For a walk-through, see the section concerning the "INI set test", section 2.7 "cfg::inimanager" on page 8.

4 Sessions

4.1 Session Naming Conventions

The *Cfg66* session support relies heavily on directories. Here are the shorthand names we will use for them; "$username" is the name of the user account; "$appname" is the short name of the application; "$clientname" is "$appname" with a distinguishing wart (e.g `-123`; "$app-version" is "$appname" with the version appended (e.g. `-0.99`.

- `$prefix`. The base installation directory of the application that uses the *Cfg66* library.
  - *Linux*: `/usr/` or `/usr/local`. Some common subdirectories:
    - `bin/`. Contains the application's executable file.
    - `include/$app-version/`. Contains the directory hierarchy of the application, to support building the application from source code.

    * `lib/$app-version/`. Contains the libraries needed for the application.
    * `man/`. Contains the application's man pages.
    * `doc/$app-version/`. Contains the application's documentation, which could include HTML text, PDF files, tutorial files, and more.
    * `share/$app-version/`. Contains stock data for the application, including icons, pixmaps, sample data files, and sample configuration files.
  - *BSD*: `/usr/local`. *FreeBSD* reserves `/usr/` for system files. Otherwise, the layout is about the same as for *Linux*.
  - *Windows*: The layout for *Windows* is notably different. `C:/Program Files/`. The user can choose a disk drive other than `C:`. We currently aim to support only 64-bit applications. Here are the common directories as set up by *Cfg66*.
    * `C:/Program Files/$appname/`. We might consider installing the executable into a versioned subdirectory of this one.
    * `C:/Program Files/$appname/data`. Contains all the installed items that are in `share/` in UNIXen.
- `~`. The HOME directory of the user as supported in various operating systems. An equivalent symbol is `$HOME`.
  - *Linux*: `/home/username/`
  - *BSD*: `/home/username/`
  - *Windows*: `C:/Users/username/`
- `$home`. The directory of the configuration files as supported by default in *Cfg66*. It can be modified by a command-line option as described in the next section.
  - *Linux*: `/home/username/.config/$appname`.
  - *BSD*: `/home/username/.config/$appname`.
  - *Windows*: `C:/Users/$username/AppData/Local/$appname`.
- `$directory`. This subdirectory can be specified in the `.session` file. Note that session files are *always* stored in *$home*, even if `$home` is modified by a command-line option. If `$directory` is not empty, it replaces `$home` as the home-directory of the application while it is running. It can have a full path if desired. If it has no path, then it is a subdirectory of `$home`.

Confused? I sure am!

4.2 Session Description

A session provides a specific configuration for an application. The environment is contained and specified in a *session directory*, also known as a *home directory*. As an example, home directories for *Seq66* running on its own or inside an *NSM* ([3]) session:

```
/home/user/.config/seq66
/home/user/NSM Sessions/Seq66/seq66.nGJDW
```

Note that the `seq66.nGJDW` directory is created by `NSM`. *Cfg66* does not create a configuration directory by default, but it has some options:

- `-home <path >`. This option changes the "home" directory if the argument includes a path, or appends a directory name to the default "home" if it does not include a path. This changes

the directory where the `.session` files are found. If `NSM` is running (as shown above), that directory becomes `$home`.
- `-session <name[.session] >`. This option selects a session file from `$home`. The default session file `$appname.session`. This option is *not* to be used if `NSM` is running.

The "home" directory could have subdirectories, such as `config`, `audio`, `midi`, `data`, etc. These are application specific, and can be listed in the `$appname.session` file.

Cfg66 supports a `.session` file that lists the names, locations, and activity of the various other INI files. With that setup, we would need a session file for each session. With `NSM` the session setup is determined by `NSM`, and the session file is always `$appname.session`. Use cases to support:

- **Standalone application**. The application automatically refers to a set of configuration files in a "home" directory, either `/home/user/.config/seq66` or either `/home/user/.config/seq66/config`.
- **Session selection**. All session information resides in a known file in the home directory, `$appname.session`, but a selection of the session file can be made at startup.
- **NSM session**. There is a session manager that communicates the session directory to the application.

5 Session Namespace

What is a session? It is an environment in which an application runs, and is one of many possible environments for the application. *Cfg66* provides rudimentary management of the characteristics of an application. Compare that to a true session manager such as the *New Session Manager*([3], NSM), which coordinates not only the main application, but a group of applications and their configuration and connections, recreating recreate complex setups based on the contents of a particular session directory.. Here, we want a way to provide various setups for an application. Of course, these setups can be placed under the control of a more complex session manager.

This section provides a useful walkthrough of the `session` namespace of the *cfg66* library. In addition, a `C`-only module is provided.

Here are the classes (or modules) in this namespace:

- `session::climanager`
- `session::configuration`
- `session::directories`
- `session::manager`

5.1 session::climanager

The `session::climanager` class is derived from `session::manager` and overrides a number of virtual functions It also provides a function to read a configuration file. It provides a `run()` loop which does nothing but check for calls to close and save the session and wait for a small polling period.

5.2 session::configuration

The `session::configuration` class is derived from `cfg::basesettings`. It contains a `session::directories` management class and a set of `directories::entries` items. Options for help, a log-file, and auto-save are provided.

Actually, using the `cfg::inimanager` seems like a better option. Stay tuned.

5.3 session::directories

The `session::directories` class manages a set of `entry` directory item.

Each `entry` specifies:

- Name of the section covered by a configuration file.
- It active/inactive status.
- The directory for the files(s).
- The base name of the file(s).
- The optional extension of the file(s).

Provides the full path specification of each file, constructed from the entries, and keyed by the section name. The "home" configuration path and the session path are also specified.

The `directories.cpp` module explains the directory layouts and provides default "rc" and "log" directory specifications.

5.4 session::manager

The `session::manager` class is base class for providing an application with "session" information, where a session is a group of configuration items that allow an application to run in a sequestered environment. Think of the *JACK Session Manager* or the *New/Non Session Manager*.

The base session manager class holds the following information:

- `session::configuration`. See the section about this class above.
- `cli::parser`. Provides access to the command-line parser.
- *Capabilities*. This application-dependent string publishes some information about the application. Useful with the *New/Non Session Manager*.
- *Manager name*. The name of the session manager. For example, it is returned by the *New/Non Session Manager*.
- *Manager path*. This item holds the directory where the session information is to be stored. For example, the *New/Non Session Manager* returns a string like `/home/user/NSM Sessions/JackSession/seq66`
- *Display name*. This is the name of the session to be displayed, such as *JackSession* in the string above.
- *Client ID*. This is the name of the client (e.g. for managing port connections), such as *Seq66* or *seq66.nUKIE*.

Also included are indicators for `-help`, dirty status, and error messages.

A large number of `virtual` members functions are included. Some of the important functions are for the following actions:

- Parsing an option (configuration) file.
- Parsing the command line.
- Creating, writing, and reading a configuration file.
- Creating, saving, and closing a session.
- Creating a session directory.
- Creating a "project".
- Creating a manager.
- Running a session, often in a loop or a GUI thread.

The `manager.cpp` module contains a short statically-initialized list of default options.

Note that there are currently a number of "To Dos" in this class.

6 Session Walkthroughs

While the session layouts that can be supported are many, we will walk through only the most common (i.e. tested) scenarios.

1. Determine the `$home` directory.
    1. Provide a `cfg::appinfo` structure that defines at least the app-name. One can also provide non-empty string values for app-version, config-section name, home directory and home configuration file, etc. If empty, defaults are used or assembled.
    2. Optionally, one can call `user_home(appfolder)` to change the folder name at run-time.
    3. Another option is to use the `-home path` option on the command-line.
    4. If a session manager is running, use the path that it provides.
2. Determine the session file. By default this is `$appname.session` in the `$home` directory. It can be changed only with the `-session` command-line option.
3. Determine the session `$home` directory. By default this is the same as `$home`.
4. Get the list of application sub-directories (for data and other files) from the session file. Make sure each directory is created or already in existence.

Note that `$home` can be set in the beginning via the CLI or function call, but with a session manager it cannot be determined until after a connection is made. Therefore, the `manager::run()` override must start with a call to `set_home()`.

TODO.

(We need a special inifile class to process a .session file.

TODO. TODO. TODO.

7 Util Namespace

This section provides a useful walkthrough of the `util` namespace of the *cfg66* library.

Here are the classes (or modules) in this namespace:

- `util::bytevector class`
- `util::filefunctions module`
- `util::msgfunctions module`

- `util::named_bools class`
- `util::strfunctions module`

All of the modules are `C++` modules with free functions in the `util` namespace.

7.1 util::bytevector

The `util::bytevector` class provides an `std::vector` of "bytes" (`unsigned char`) with functions to put bytes into the vector and read them out. There are also functions to read a file and write the vector to the file.

The bytes are treated as a stream of *big-endian* data. Big endian is a storage format where the most significant byte (MSB) of a multibyte data value is stored at the lowest memory address, "big-end-first." It is also called "network byte order" because it is the standard for network protocols. It facilitates easier human readability of hexadecimal dumps and allows numerical sorting of byte string. Also note that MIDI data is big-endian.

Integers are extracted from the bytes a byte at a time, starting with the most significant byte. Since this data is big-endian, it is suitable for use with MIDI files and network data streams.

This module defines some types in the `util` namespace:

- `byte` = `uint8_t`
- `ushort` = `uint16_t`
- `ulong` = `uint32_t`
- `ulonglong` = `uint64_t`
- `bytes` = `std::vector<byte>`.

The `util::bytes` type holds the big-endian data. The `util::bytevector` class keeps track of the section of data being processed, and the position in the data. In addition, errors are detected, and an error message is stored for later display.

Various overloads of the `assign` member function are used for loading data into the byte-vector. Functions are provided to "get" and "peek" at items in the byte-vector. The former change the position value, while the latter do not. Functions are provided to "put" and "poke" items into the byte-vector. The former change the position value, while the latter do not.

Lastly, "read" and "write" functions are provided for interactions with a data file.

7.2 util::filefunctions module

The `util::filefunctions` module contains a large number of function dealing file-names and files. The file-name functions are useful for building paths and for splitting paths into parts. The file functions are basically wrappers around the `C FILE *` API. Also provided are functions to check file attributes, to read/write strings, and to open, close, copy, and append data.

Really, just skim the `filefunctions` modules to learn what is there. They include every convenience function we needed in implementing *Seq66*.

7.3 util::msgfunctions module

The `util::msgfunctions` module defines functions for writing messages to the console along with tags showing the short name of the application that wrote them, and in color represent whether the message is an error, warning, debug, status, file-related, or session message.

Also included are some "async safe" functions for output and for converting unsigned numbers to string arrays.

Setters and getters are provided for the `-verbose` and `-investigate` options of the command-line parser.

Again, skim the `util::msgfunctions` module for details.

7.4 util::named_bools

The `util::name_bools` class makes it easy to look up and set a "small" number of boolean values by name.

This class could be useful if one does not want the full capability of the options-related classes in the `cfg` namespace.

7.5 util::strfunctions module

The `util::strfunctions` module defines functions for manipulating strings: tokenization, left/right space trimming, conversion between strings and values with the added feature of defaulting, word-wrapping, and formatting of `std::string` values without using `std::stringstream`.

Skim the `util::strfunctions` module for details.

8 Cfg66 Tests

This section provides a useful walkthrough of the testing of the *cfg66* library. They illustrate the various ways in which the *Cfg66* library can be used by a developer.

The tests so far are these executables:

- `cliparser_test_c`
- `cliparser_test`
- `history_test`
- `ini_test`
- `manager_test`
- `options_test`

These tests are supported by data structures define in the following header files:

- textttctrl_spec.hpp
- textttdrums_spec.hpp
- texttttmutes_spec.hpp
- textttpalette_spec.hpp

- textttplaylist_spec.hpp
- texttrc_spec.hpp
- textttsession_spec.hpp
- texttttest_spec.hpp
- texttusr_spec.hpp

These header files will be discussed as needed in the following sections.

8.1 Cfg66 cliparser_test_c Test

This test of the `C` command-line interface uses the free functions in `cliparser_c.h`. The test module itself sets up a small set of test options:

```
static options_spec s_test_options [] =
{
    //   Name          Code  Kind     Enabled   Default     Value      Dirty
    {
        "alertable",    'a', "boolean", true,   "false",    "false",   false,
        "If specified, the application is alertable."
    }, . . .
};
```

The test makes changes to the options and verifies that they took hold. The test command is simple:

```
$ ./build/test/cliparser_test_c
```

It shows the changes and a result statement.

8.2 Cfg66 cliparser_test Test

This test uses the following tests options (only one is shown) static initialization:

```
static cfg::options::container s_test_options
{
    //   Name, Code,  Kind, Enabled,
    //   Default, Value, FromCli, Dirty,
    //   Description, Built-in
    {
        {
            "alertable",
            {
                'a', "boolean", cfg::options::enabled,
                "false", "false", false, false,
                "If specified, the application is alertable.",
                false
            }
        }, . . .
    }
};
```

It serves as a good example of how to create a list of options. More flexible than GNU's getopt setup and simplifies generating help text.

The test makes changes to the options and verifies that they took hold. The test command is not as simple as the `C` version, as verbosity is needed to see the changes:

```
$ ./build/test/cliparser_test --verbose
```

It shows the changes and a result statement. This test needs a little bit of cleanup.

8.3 Cfg66 history_test Test

The history test also sets up a test options "array". Then it makes changes to the options, such as changing variables, undoing the change, and redoing the change. It shows the changes and a result statement.

See this test file for some "To do" items.

8.4 Cfg66 ini_test Test

This test program uses all of these "data" headers:

- textttctrl_spec.hpp
- textttdrums_spec.hpp
- texttttmutes_spec.hpp
- textttpalette_spec.hpp
- textttplaylist_spec.hpp
- texttttrc_spec.hpp
- textttsession_spec.hpp
- texttttusr_spec.hpp

It defines these static test items:

- `cfg::options::container s_test_options`. This sets up a single option called "test", used as a command-line option.
- `cfg::inisection::specification s_simple_ini_spec`. This sets up an `[experiments]` section with a number of option-variable definitions.
- `cfg::inisection::specification s_section_spec`. This sets up an `[section-test]` section.
- `cfg::inifile::specification exp_file_data` This items sets up the "experiment" configuration directory using `cfg::inisection::specification s_simple_ini_spec`. `cfg::inisection::specifi s_section_spec`.

Additional sections are defined and add to a `cfg::inifile::specification` declaration.

Hmmm. some are unsued.

8.5 Cfg66 manager_test Test

This test defines `cfg::appinfo s_application_info`. This is used here: `cfg::initialize_appinfo(s_applicati argv[0])`.

The "To do" here is to actually implement `simple_smoke_test`.

8.6 Cfg66 options_test Test

This test program uses only the "data" header `test_spec.hpp` which defines `cfg::options::container s_test_options`. This container is used to initialize a `cli::parser`. That object then gets the command-line arguments.

Obviously, we still have a lot of work to do with these tests.

9 Summary

Contact: If you have ideas about *Cfg66* or a bug report, please email us (at `mailto:ahlstromcj@gmail.com`).

10 References

The *Cfg66* references list.

References

[1] Twinkle Sharma. *Deque in C++* https://www.scaler.com/topics/cpp/deque-in-cpp/. 2024.

[2] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Use your search engine. Start with page 62. 1994.

[3] JACK Audio. *New Session Manager, an offshoot of Non Session Manager*. https://github.com/jackaudio/new-session-manager 2021.

[4] Chris Ahlstrom. *A reboot of the Seq24 project as "Seq66"*. https://github.com/ahlstromcj/seq66/. 2015-2024.

Index